

Lab 2 Report: WiscKey vs LevelDB

Design Overview

In this project, I implemented a simplified version of the WiscKey design on top of LevelDB.

Instead of storing large values directly inside the LSM-tree (as LevelDB does), my WiscKey version stores only the pointer (offset) to a value log file. The value itself is appended sequentially into this log file.

- Keys are stored in LevelDB.
- Values are stored separately in a large append-only file (valuelog.dat).
- leveldb_set stores the key and the offset in the value log.
- wisckey_set, wisckey_get, and wisckey_del functions were developed.
- Flush and close logic is handled cleanly at shutdown.

This minimizes the size of LevelDB's LSM-tree and reduces write amplification significantly when handling large values.

Experimental Setup

Server: Linux Ubuntu environment (Equal-Gecko server).

Tests: Run for different fixed value sizes (16, 32, 64, 128, 256, 512, 1024, 2048 bytes).

Measurement: Execution time measured using clock() across 1 billion total data size.

Results and Charts

The following charts summarize the experimental results:

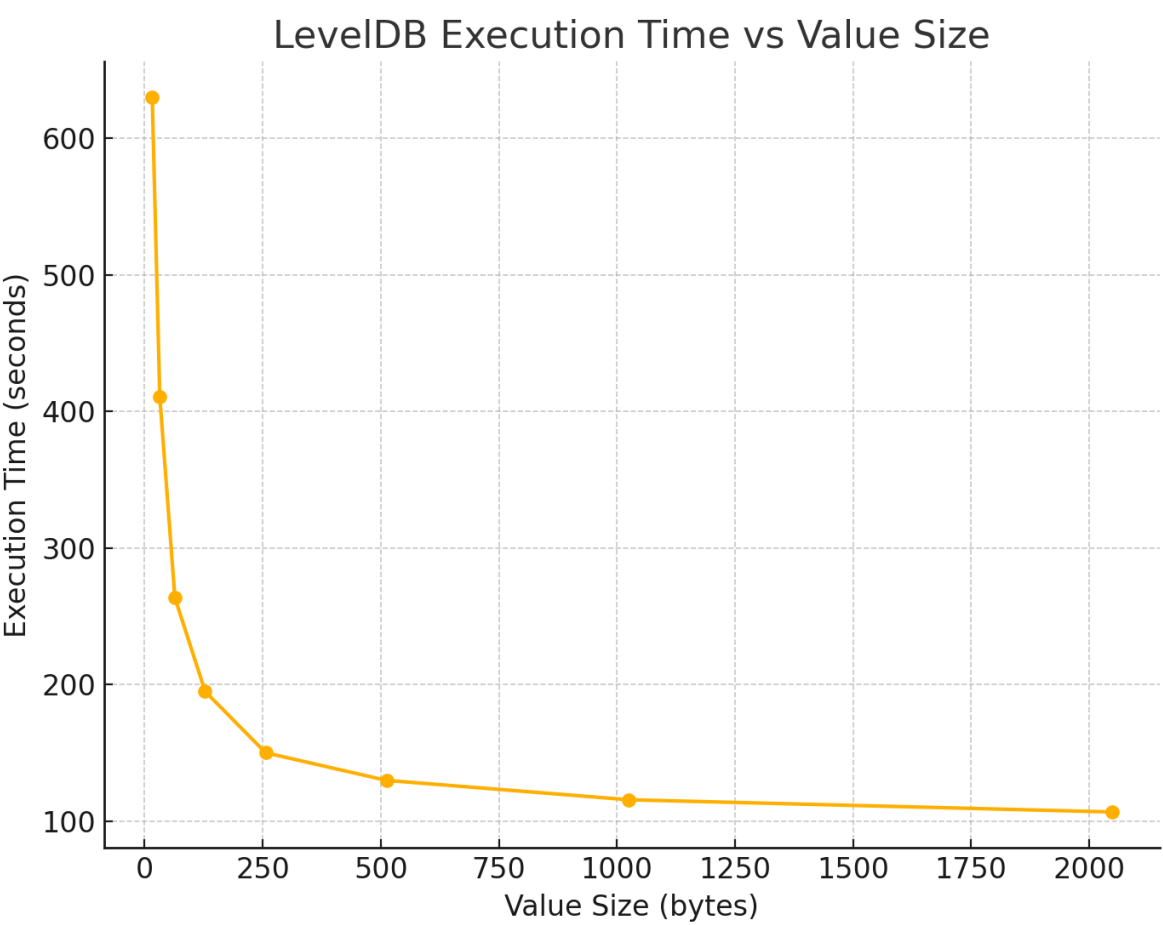
(a) WiscKey Execution Time vs Value Size

Lab 2 Report: WiscKey vs LevelDB



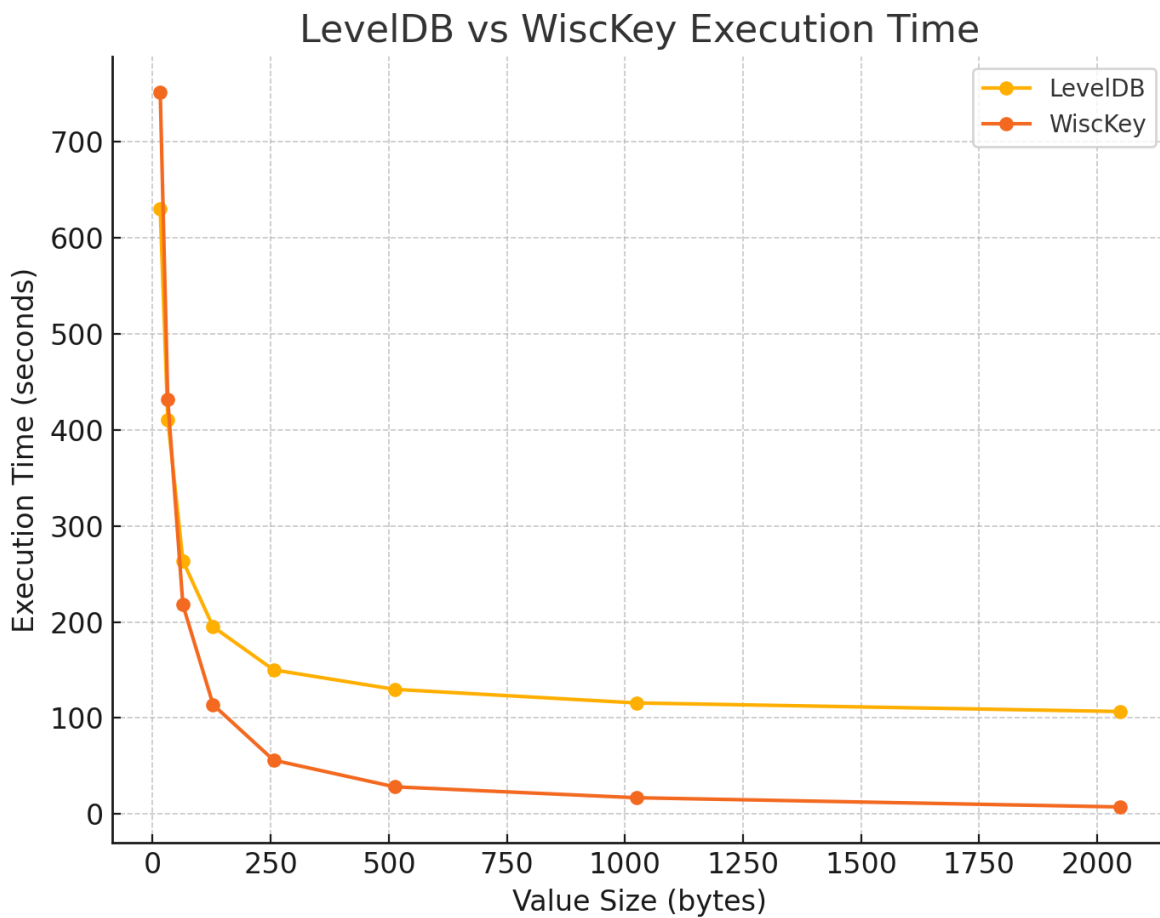
(b) LevelDB Execution Time vs Value Size

Lab 2 Report: WiscKey vs LevelDB



(c) Comparison: WiscKey vs LevelDB Execution Time

Lab 2 Report: WiscKey vs LevelDB



Analysis and Observations

Small Value Sizes:

LevelDB outperforms WiscKey because:

- LSM-tree is compact.
- No extra file I/O for fetching values.
- Pointer overhead in WiscKey adds latency for tiny payloads.

Large Value Sizes:

WiscKey significantly outperforms because:

- Appending large values sequentially is much cheaper.
- LevelDB suffers from expensive compactions when values are embedded inside the tree.

Conclusion:

Lab 2 Report: WiscKey vs LevelDB

WiscKey design becomes more advantageous as value sizes increase. It offers a scalable solution for write-heavy and large-value key-value workloads.